Best Practices Jupyter

Jupyter Notebooks

Jupyter notebooks provide a way to run code in an interactive environment. While most prominently used for Python, Jupyter supports a range of languages, such as Julia and R.

Choosing an Approach

There are multiple ways to run Jupyter notebooks on Engaging. The route you choose depends on your needs and level of familiarity with high performance computing environments.

OnDemand Web Portal

The most straightforward way to run a Jupyter notebook is to use Engaging's web portal.

- Link to web portal: https://orcd-ood.mit.edu/
- Select "Interactive Apps" --> "Jupyter Notebook"
- Fill out the form to request a time limit for your job, number of cores, memory, and Jupyter kernel. The kernel corresponds to the base set of packages that we have installed to the respective Miniforge module.
- Follow the on-screen instructions to start a session. When the session is ready, click "Connect to Jupyter." From here you can create a Jupyter notebook and select the language or kernel you would like to use.
- If you want to use a custom Conda environment, you will need to run the following with your Conda environment activated (after installing `jupyterlab`):

```
python -m ipykernel install --user --name $CONDA_DEFAULT_ENV
```

- You may need to close your notebook and open it again for the new kernel to be visible.

VS Code

First, follow these instructions to set up VS Code to run **on a compute node**.

Open a Jupyter notebook and click the top-right button to select a kernel. You can select "Python Environments" for any Conda environments or "Jupyter Kernel" to find Julia or R environments. If you have installed R with Conda, you can find your Conda environment under "Jupyter Kernel." `jupyterlab` must be installed to your Conda environment.

Language-Specific Instructions

Julia

You will need to add the `IJulia` package to your environment for Jupyter to recognize the Julia kernel. You can do so from the command line:

Bash

```
module load julia
julia
```

Julia

```
using Pkg
Pkg.add("IJulia")
Pkg.build("IJulia")
```

Unlike R, Julia environments are separate from Conda. However, if the `IJulia` package is installed, then the Julia kernel should be visible regardless of the Conda environment from which you run your Jupyter notebook.

See our Julia documentation for more information.

Python

To run Python Jupyter notebooks, install `jupyterlab` in whatever Conda environment that contains the packages you need.

See our Python documentation for more information.

R

To run R in a Jupyter notebook, you need to create a Conda environment with both `r-irkernel` and `jupyterlab` installed:

```
module load miniforge
conda create -n r_jupyter_env jupyterlab r-irkernel
conda activate r_jupyter_env
```

Most R packages are available through Conda, so feel free to install other packages you need in this environment.

See our R documentation for more information.

FAQs

How do I run a Jupyter notebook with a GPU?

The ORCD OnDemand web portal offers an option to allocate a GPU to your Jupyter session.

Jupyter does not recognize the kernel for my environment. What do I do?

First, make sure you have `r-irkernel` installed if you're using R, `IJulia` installed (and built) if you're using Julia, and `jupyterlab` installed in your Conda environment.

On VS Code, you may need to specify the path to the `conda` binary of the Conda installation you're using. This can be done by editing the "Python: Conda Path" setting. For example, if you're using the `miniforge/24.3.0-0` module on Engaging, the path would be:

```
/orcd/software/core/001/pkg/miniforge/24.3.0-0/condabin/conda
```

To see all kernels that Jupyter recognizes, activate a Conda environment with `jupyterlab` installed and run `jupyter kernelspec list`.

I tried to install `jupyterlab` in my Conda environment, but the installation failed. How can I run a Jupyter notebook with the dependencies I need?

It's best to install the packages you need when you create a Conda environment rather than one-by-one after the environment has been created. This will make Conda more likely to solve your environment successfully. For example:

```
conda create -n jupyter_env jupyterlab pandas pytorch
```

See Conda Environments for more information.

How do I access my lab's storage from Jupyter?

By default, Jupyter treats the root directory as whatever directory you started Jupyter from. If you're using the OnDemand web portal, this is your home directory. This effectively blocks you from navigating to other paths, such as `/orcd/data` or `/orcd/pool`, where many groups have dedicated shared storage.

To get around this, you can create a symlink in your home directory:

```
ln -s /path/to/destination /path/to/symlink
```

April 29, 2026